



DEPI GRADUATION PROJECT

LAND TYPE

CLASSIFICATION

PRESENTED BY OUR TEAM...

01	PROJECT BRIEF	05	MODEL ARCHITECTURE
02	ABOUT DATA	06	TRAINING AND VALIDATION
03	EXPLORATORY DATA ANALYSIS	07	CONCLUSION
04	DATA PREPROCESSING		

CONTENT

PROJECT OVERVIEW

- Objective:
 - Classify satellite images into different land use and land cover categories (e.g., Forest, Residential, River, etc.) using machine learning and deep learning techniques.
- Dataset:
 - Source: EuroSAT — based on Sentinel-2 satellite imagery.
 - Structure: Images are organized into folders, each representing one of 10 land use classes.
 - Format: RGB and/or multispectral .jpg/.tif images, size usually 64x64.

Mohamed Reda

WHAT WE DID (PROJECT STEPS):

1. EDA (Exploratory Data Analysis):

- a. Visualize image samples from each class.
- b. Plot image distribution per class.
- c. Analyze image shapes, sizes, and color intensities.
- d. Check balance of the dataset.

2. Preprocessing:

- a. Resize/normalize images.
- b. Optional: Convert to grayscale or reduce channels.
- c. Data augmentation (e.g., flipping, rotation).

3. Model Building:

- a. Baseline: Use traditional ML (like SVM, RandomForest) on flattened image features (optional).
- b. Deep Learning: Use CNN (Convolutional Neural Networks) with PyTorch/Keras/TensorFlow.
- c. Optional: Transfer learning using pretrained models like ResNet or VGG.

ABOUT DATA:



Classes:

- AnnualCrop
- Forest
- HerbaceousVegetation
- Highway
- Industrial
- Pasture
- PermanentCrop
- Residential
- River
- SeaLake

What is it?

EuroSAT is a labeled satellite imagery dataset based on Sentinel-2 satellite data, designed for land use and land cover classification. It's perfect for training models to understand Earth's surface patterns.

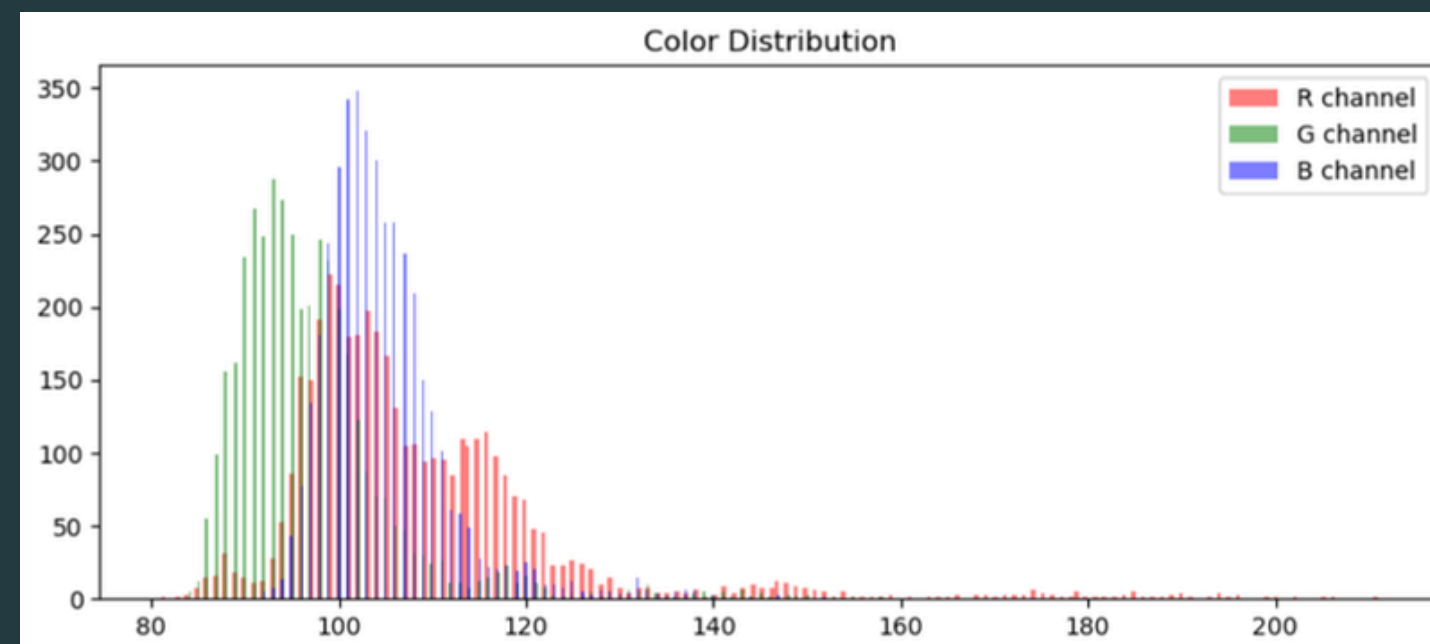
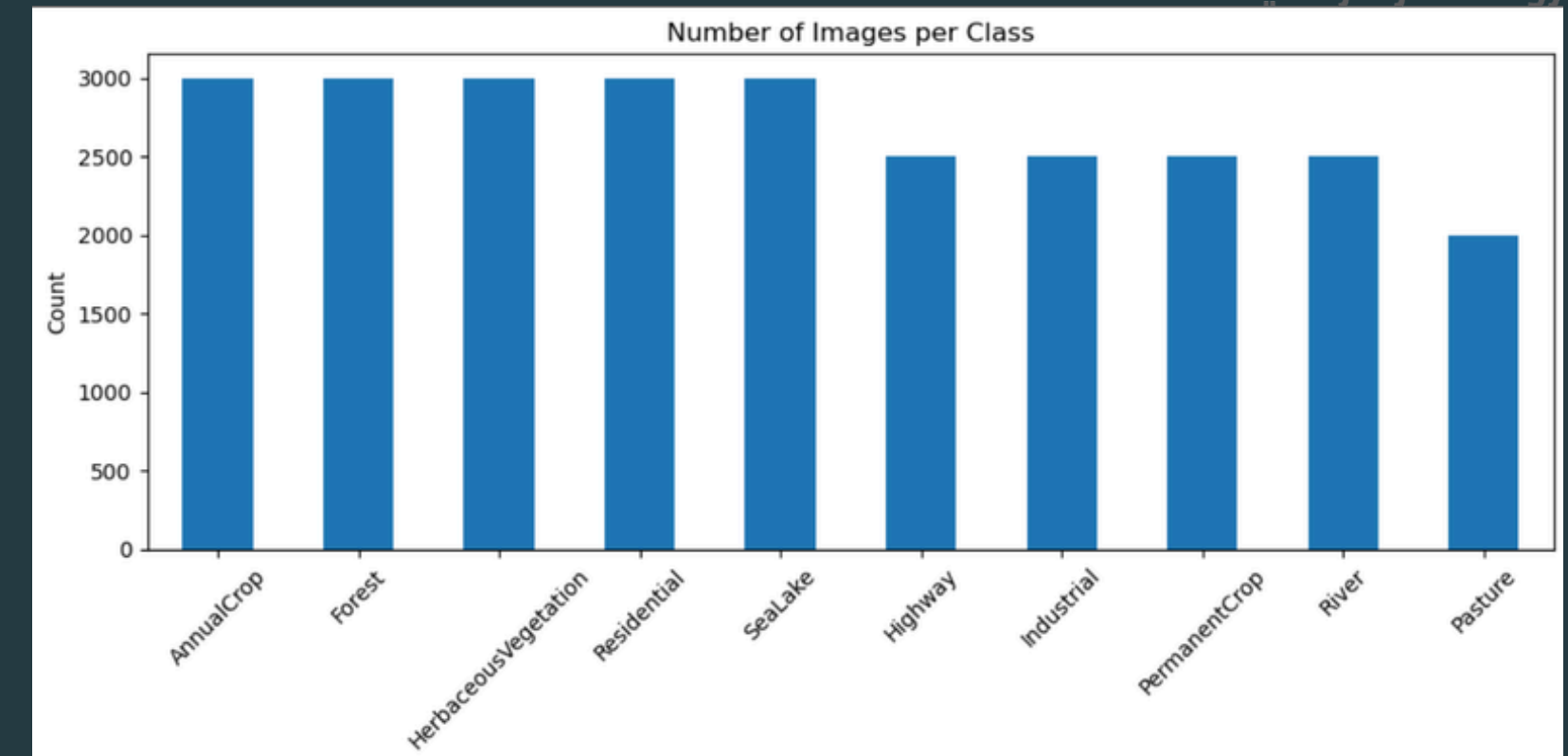
Data Stats:

- Total Images: ~27,000+
- Images per Class: ~2,000 to 3,000 (almost balanced)
- Resolution: Medium — not super detailed, but great for ML tasks.

EXPLORATORY DATA ANALYSIS (EDA):



- Class Distribution Summary:
 - Highly imbalanced dataset
 - Top classes: "Annual/CRP" (2500-3000), "Forest" (2000-2500)
 - Bottom classes: "Highway", "Industrial", "Pasture" (<500 each)
- Key Concern:
 - Model may struggle with rare classes due to extreme imbalance.
- Solution:
 - Apply class weights or oversample rare classes during training.



- Shows RGB channel histograms from a sample image.
- Reveals pixel intensity distribution (0-255) per channel.
- Typical natural image profile with:
 - Peaks in mid-range values
 - Long tails at extremes
- Helps detect color bias.
- Useful for checking dataset color characteristics before augmentation/normalization.

DATA PREPROCESSING:

This function creates a ImageDataGenerator with two modes:

1. Augmentation Mode (default):

- Applies 8 transformations: rotation, shifts, shear, zoom, flip, brightness adjustment
- Normalizes pixels (1./255)
- Uses 20% data for validation
- Fills empty pixels with nearest neighbor

2. Basic Mode

- Only performs pixel normalization

Purpose: Enhances training data variety to improve model generalization.

```
def create_datagen(augmentation=True):  
    """Create data generators with enhanced augmentation"""  
    if augmentation:  
        train_datagen = ImageDataGenerator(  
            rescale=1./255,  
            rotation_range=40,  
            width_shift_range=0.2,  
            height_shift_range=0.2,  
            shear_range=0.2,  
            zoom_range=0.3,  
            horizontal_flip=True,  
            brightness_range=[0.7, 1.3],  
            fill_mode='nearest',  
            validation_split=0.2  
        )  
    else:  
        train_datagen = ImageDataGenerator(rescale=1./255)  
  
    return train_datagen
```


DATA PREPROCESSING (CONT...):

This function creates three data generators:

1. Training Generator:

- Uses augmented data (rotations, flips, etc.).
- 80% of data (specified by `subset='training'`).
- Batched for efficient training.

2. Validation Generator:

- Same augmentation as training (prevents mismatch).
- 20% of data (subset='validation').

3. Test Generator

- No augmentation, batch size = 1 (for accurate evaluation)
- Disables shuffling for consistent results

Key Features:

- ✓ Enforces reproducibility with fixed `seed`
- ✓ Saves class-to-index mapping for deployment
- ✓ Supports custom class order via `CLASS\_NAMES`

```
def get_data_generators():  
    """Create train/validation/test generators"""  
    train_datagen = create_datagen(augmentation=True)  
    test_datagen = create_datagen(augmentation=False)  
  
    train_generator = train_datagen.flow_from_directory(  
        DATA_PATH,  
        target_size=IMG_SIZE,  
        batch_size=BATCH_SIZE,  
        class_mode='categorical',  
        subset='training',  
        seed=SEED,  
        classes=CLASS_NAMES  
    )  
  
    val_generator = train_datagen.flow_from_directory(  
        DATA_PATH,  
        target_size=IMG_SIZE,  
        batch_size=BATCH_SIZE,  
        class_mode='categorical',  
        subset='validation',  
        seed=SEED,  
        classes=CLASS_NAMES  
    )  
  
    test_generator = test_datagen.flow_from_directory(  
        DATA_PATH,  
        target_size=IMG_SIZE,  
        batch_size=1,  
        class_mode='categorical',...
```


Model Architecture:

Core Structure:

- Input Preprocessing: Built-in normalization (+ random rotation/zoom augmentation)
- Convolutional Base:
 - 3 Conv2D blocks (64 → 128 → 256 filters) with BatchNorm and max pooling
 - GlobalAveragePooling for dimensionality reduction
- Classifier:
 - 512-unit Dense layer with Dropout (50%)
 - Softmax output for multi-class prediction
- Key Features:
 - ✓ Built-in augmentation (rotation + zoom)
 - ✓ Batch normalization for stable training
 - ✓ Global pooling reduces parameters vs Flatten
 - ✓ Adam optimizer (LR=1e-3) with categorical crossentropy

```
def create_model():
    """Create enhanced CNN model with preprocessing layers"""
    model = models.Sequential([
        # Input preprocessing
        preprocessing.Rescaling(1./255, input_shape=(*IMG_SIZE, 3)),
        preprocessing.RandomRotation(0.1),
        preprocessing.RandomZoom(0.1),

        # Convolutional Base
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        layers.GlobalAveragePooling2D(),

        # Classifier
        layers.Dense(512, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(len(CLASS_NAMES), activation='softmax')
    ])

    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model
```

Training and Validation:

- EarlyStopping: Halts training if no improvement after 5 epochs (restores best weights)
- ModelCheckpoint: Saves best model automatically ('best_model.keras')
- ReduceLROnPlateau: Cuts learning rate by 90% after 3 stagnant epochs
- TensorBoard: Enables real-time training visualization

```
def get_callbacks():  
    """Create and return a list of training callbacks"""  
    return [  
        callbacks.EarlyStopping(patience=5, restore_best_weights=True),  
        callbacks.ModelCheckpoint('best_model.keras', save_best_only=True),  
        callbacks.ReduceLROnPlateau(factor=0.1, patience=3),  
        callbacks.TensorBoard(log_dir='./logs')  
    ]
```

```
if __name__ == "__main__":  
    # Training  
    print("\nStarting model training...")  
    history = model.fit(  
        train_gen,  
        epochs=EPOCHS,  
        validation_data=val_gen,  
        callbacks=get_callbacks()  
    )  
    print("\nTraining completed!")
```

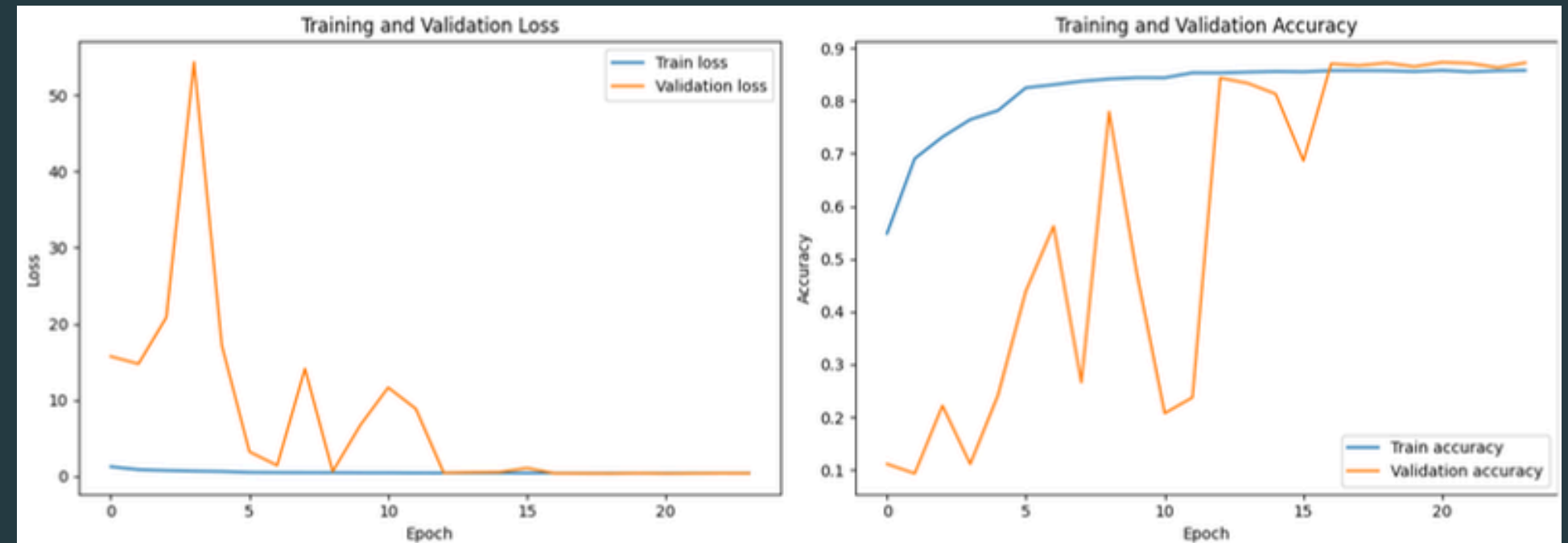
- Launches model training with:
 - Epochs: Custom EPOCHS setting
 - Data: Uses train/validation generators
 - Callbacks: Applies automated optimization (early stopping, checkpoints, LR adjustment)
 - Progress: Prints start/end notifications
- Output: Returns history object containing all training metrics.

- Executes critical test-phase operations:
 - Rigorous Testing: Evaluates model on untouched test set
 - Key Metrics: Calculates loss and accuracy (4 decimal precision)
 - Clear Reporting: Prints formatted results for quick interpretation
- Output: Returns exact performance metrics to gauge real-world applicability.

```
if __name__ == "__main__":  
    # Evaluation  
    print("\nEvaluating on test set...")  
    test_loss, test_acc = model.evaluate(test_gen)  
    print(f"\nTest Accuracy: {test_acc:.4f}")  
    print(f"Test Loss: {test_loss:.4f}")
```

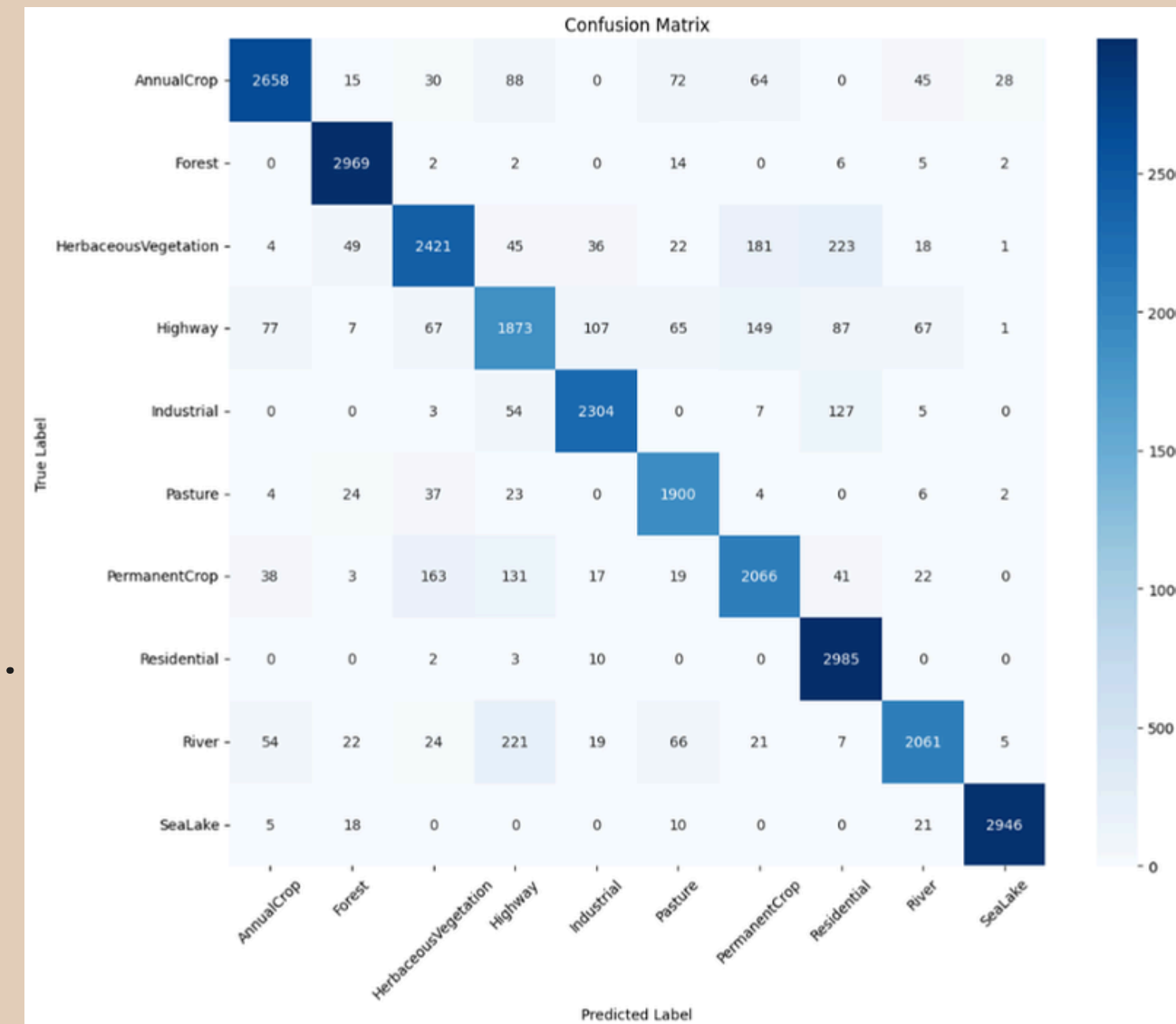
Training and Validation:

- **Loss Curve:**
 - Both training and validation loss decrease steadily
 - No significant divergence, suggesting no severe overfitting
 - Final loss values near zero indicate good convergence
- **Accuracy Curve:**
 - Training accuracy reaches ~90% (strong learning)
 - Validation accuracy stabilizes around ~80%
 - Small gap between curves shows moderate generalization
- **Key Insight:**
 - Model learns effectively but may benefit from:
 - Regularization to reduce train-val gap
 - Early stopping around epoch 15



Training and Validation:

- Confusion Matrix:
 - The confusion matrix provides a detailed breakdown of how well the trained model performs on the test dataset by comparing true labels to predicted labels.
 - Rows represent the actual classes (ground truth).
 - Columns represent the predicted classes.
 - Each cell $[i, j]$ shows the number of times images from class i were predicted as class j .
- Insights It Offers:
 - Diagonal cells show correct predictions — the higher these values, the better.
 - Off-diagonal cells show misclassifications — helping you spot which classes the model confuses.



Conclusion

-  Successfully trained a deep learning model to classify satellite images into multiple land cover categories using the EuroSAT dataset.
-  Performed exploratory data analysis (EDA) to understand class distribution and visualize sample images from each class.
-  Evaluated model performance using a confusion matrix, identifying both strengths and common misclassification patterns.
-  Data augmentation and normalization helped improve generalization and model robustness.
-  The project demonstrates the potential of AI in geospatial analysis and remote sensing applications, supporting smarter land use and environmental monitoring.

TEAM MEMBERS

Team Members:

- Mohamed Reda ALi
- Morad Asker Hassan
- Ayman Mohamed El-desoky
- Samy Mostafa El-yazory
- Abdelrahman Ahmed Mahmoud
- Ahmed Wagdy Nasr

THANK YOU

Feel free to give feedback and input

m07hamedreda25@gmail.com

+201068145227

